

Intermediate (timeouts and retry-with-backoff) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — a plain retry loop

```
# retry_backoff_practice.py – Intermediate section
import time
import requests

# part 1: prove the timeout fires
try:
    requests.get("http://10.255.255.1", timeout=(1, 2))
except requests.exceptions.Timeout:
    print("timed out, as expected")

# part 2: retry loop with backoff
max_attempts = 5
response = None

for attempt in range(max_attempts):
    try:
        response = requests.get("https://api.github.com", timeout=(3, 5))
        break
    except requests.exceptions.Timeout:
        print("attempt", attempt, "timed out")
        if attempt == max_attempts - 1:
            print("gave up after", max_attempts, "attempts")
        else:
            time.sleep(2 ** attempt)

if response is not None:
    print(response.status_code)
```

Expected output (the real call succeeds on attempt 0, since `api.github.com` responds fine):

```
timed out, as expected  
200
```

This works. It's missing type hints and re-raises nothing — a caller of this script-level code can't distinguish "it worked" from "it gave up" except by checking `response is not None` themselves.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a typed, reusable function that re-raises

```
# retry_backoff_practice.py – Intermediate section
import time
import requests

def get_with_retry(url: str, max_attempts: int = 5) -> requests.Response:
    for attempt in range(max_attempts):
        try:
            return requests.get(url, timeout=(3, 5))
        except requests.exceptions.Timeout:
            if attempt == max_attempts - 1:
                raise
            wait_seconds = 2 ** attempt
            print(f"attempt {attempt} timed out, waiting {wait_seconds}s")
            time.sleep(wait_seconds)
    raise RuntimeError("unreachable") # loop always returns or raises above

if __name__ == "__main__":
    try:
        requests.get("http://10.255.255.1", timeout=(1, 2))
    except requests.exceptions.Timeout:
        print("timed out, as expected")

    response = get_with_retry("https://api.github.com")
    print(response.status_code)
```

Expected output:

```
timed out, as expected
200
```

Difference from Basic: pulling the loop into `get_with_retry()` means a caller gets back either a real `Response` or a real, propagated `requests.exceptions.Timeout` — never a silent `None`

they have to remember to check. Type hints (`url: str` , `-> requests.Response`) also turn the function signature into documentation.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — jitter, and the backoff math split into its own testable function

```
# retry_backoff_practice.py – Intermediate section
import random
import time
import requests

def compute_backoff_delay(attempt: int) -> float:
    """Exponential backoff with jitter: 2**attempt seconds, plus a small
    random extra so many failing callers don't retry in lockstep."""
    return (2 ** attempt) + random.uniform(0, 1)

def get_with_retry(url: str, max_attempts: int = 5) -> requests.Response:
    last_error: Exception | None = None

    for attempt in range(max_attempts):
        try:
            return requests.get(url, timeout=(3, 5))
        except requests.exceptions.Timeout as e:
            last_error = e
            if attempt < max_attempts - 1:
                time.sleep(compute_backoff_delay(attempt))

    raise TimeoutError(f"gave up after {max_attempts} attempts") from last_error

if __name__ == "__main__":
    print(compute_backoff_delay(3))          # roughly 8.something, no network call
    response = get_with_retry("https://api.github.com")
    print(response.status_code)
```

Expected output:

```
8.417...
200
```

The first line proves `compute_backoff_delay(3)` works entirely on its own, with no server, no `time.sleep`, and no mocking — that's the whole point of separating it from the loop.

Approach 2 — skip writing the loop at all: `urllib3`'s built-in `Retry`

```
# retry_backoff_practice.py – Intermediate section
import requests
from requests.adapters import HTTPAdapter
from urllib3.util import Retry

def make_session_with_retries(max_attempts: int = 5) -> requests.Session:
    retry_strategy = Retry(
        total=max_attempts,
        backoff_factor=1, # roughly 1s, 2s, 4s, 8s... between attempts
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["GET", "PUT", "DELETE", "OPTIONS", "HEAD"],
    )
    session = requests.Session()
    session.mount("https://", HTTPAdapter(max_retries=retry_strategy))
    session.mount("http://", HTTPAdapter(max_retries=retry_strategy))
    return session

session = make_session_with_retries()
response = session.get("https://api.github.com", timeout=(3, 5))
print(response.status_code)
```

Expected output:

```
200
```

`urllib3.util.Retry` (the library `requests` itself is built on) already implements attempt limits, exponential backoff, and status-code-based retrying — you configure it once on a `Session`, and every call through that session retries automatically, with no loop in your own code at all. It doesn't include jitter by default, and it doesn't distinguish "worth retrying" as flexibly as your own `is_transient_status()`-style function could (that flexibility is exactly what this document's Build Task needs), but for many real projects, this is genuinely less code to maintain than a hand-written loop.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's loop is correct but retries every failing caller at exactly the same moments, and its backoff math is buried inside the loop where it can't be tested without a real (or mocked) network call. Approach 1 fixes both: jitter breaks the lockstep-retry problem, and `compute_backoff_delay()` becomes a pure

function you can test with a plain assertion. Approach 2 is the "don't write this yourself" option — it hands attempt-counting and backoff to a well-tested library, at the cost of less control over exactly which failures count as retryable and no built-in jitter.

Which one should you actually write? For this document's Build Task, Approach 1's pattern — a small, pure `compute_backoff_delay(attempt)` function, separate from the loop that touches the network — is what you want, because the Build Task's constraint is explicit: your retry code needs to be testable without making real network calls, and `is_transient_status()`-style classification (timeout vs. 5xx vs. 429 vs. a real 4xx) needs to be more specific than `urllib3`'s `status_forcelist` easily allows. Reach for Approach 2 in a smaller project where "retry a handful of status codes with backoff" is genuinely all you need, and you'd rather not maintain that loop by hand at all.